Spencer Collins

Intro To C Programming

Assignment 6

6/22/2026

6.1 Does replacing gets with fgets as above fix all security issues in main on page 1-2? Why or why not?

Replacing gets() wtih fgets() fixes all the security issues in main on pages 1-2. Fgets() allows the developer to limit the length of the input from the user. In this case fgets() would limit the username to 16 characters for s.name which would prevent a buffer overflow into the memory space that s.ok is occupying.

6.2. An often-seen formulation of int_compar above is:

```
int int_compar(const void *p1, const void *p2) {
  int x1 = *((const int *) p1);
  int x2 = *((const int *) p2);

  return x1 - x2;
}
```

It seems like this might be better, insofar as it uses one operation and no branching, as opposed to two comparisons. Why isn't it?

This is not better because it can lead to a buffer overflow if the difference between x1 and x2 is large enough. It is better to use branching in this function to avoid buffer overflows.

6.3. Pick any high-level language you've used for any project of appreciable size. Tell me something you think the runtime uses function pointers for, and why. What do you believe is gained, and what is lost, due to this decision? You don't have to be right; just give me your best guess.

I think the Python runtime uses function pointers for storing functions inside of dictionaries. If two functions have the same signature in the dictionary, then the I think the interpreter would only be able to use the pointers to reference the same function signature without having to construct each function individually. I think that the Python interpreter would not be able to optimize these functions ahead of time when it is interpreting the Python code.

6.4. Predict what this program will do. (You will not be graded on the correctness of your prediction.) Then run it. Was your prediction correct? Why or why not?

```
#include <inttypes.h>
#include <stdio.h>
```

```
int main () {
    uint8_t a = 37;
    uint8_t b = 38;
    int c = (int) (uint8_t) (a - b);
    int d = (int) (a - b);

    if (c == d) {
        printf("equal\n");
    } else {
        printf("not equal\n");
    }

    return 0;
}
```

This program initializes a and b as unsigned 8 bit integers. The variable, c, is cast as an integer that is assigned a value of (a-b) that is cast to an unsigned 8 bit integer and then to an integer. The variable, d, is cast as as integer that is assigned a value of (a-b) that is cast to an integer. The program will then compare the values of c and d and then print whether c and d are equal or not and exit with a return code of 0. My prediction is that the function will print “not equal”), since c will be 1 and d will be -1. My prediction on the program’s output was correct, but the predicted value of c was wrong. Since uint8_t values can only be 0 to 255, (a-b) is -1 and becomes 255 when cast to a uint8_t. Thus, 255 != -1, so not equal.